Solución Taller — Pilas, Colas y Listas Enlazadas

Asignatura: Estructura de Datos — Institución Universitaria Pascual Bravo

Samuel Bermudez Rua

Parte 1 — Reconocer la estructura adecuada

1. **Un editor de texto guarda las acciones recientes para permitir deshacer primero la última acción realizada:**

   Estructura: Pila.

   Explicación: Funciona igual que una torre de platos: lo último que ponemos arriba es lo primero que sale. entonces como necesitamos eliminar lo ultimo que hacemos para esto nos sirve una pila.

2. **Las personas esperan su turno en una fila de atención y se atiende primero a quien llegó primero:**

   Estructura: Cola.

   Explicación: Trabaja igual que una fila por ejemplo en un banco. El primero que llega a la fila es el primero en ser atendido y salir

3. **Una aplicación recorre las canciones de una lista y, al terminar la última, vuelve automáticamente a la primera:**

   Estructura: Lista circular.

   Explicación: En esta estructura el último nodo no apunta a algo vacío si no que se conecta otra vez con el primero y asi es que se vuelve una lista circular o continua

4. **Un sistema necesita recorrer una colección de registros en un único sentido y agregar elementos al comienzo con frecuencia:**

   Estructura: Lista simple (o lista enlazada simple).

   Explicación: Solo necesitamos movernos hacia adelante cuando vamos a Insertar al principio es facil, porque solo tenemos que conectar en nodo nuevo con la cabeza actual, y no tenemos que mover las otras cosas.

5. **Un reproductor permite pasar a la canción siguiente y a la anterior desde cualquier canción:**

Estructura: Lista doble (o lista doblemente enlazada).

Explicación: Cada elemento guarda dos enlaces uno para saber cuál es la canción que va a sonar despues y el otro es para saber cuál estaba antes por eso es que nos podemos mover adelante o atras estando en cualquier punto.

Parte 2 — Pilas (LIFO)

2.1 Sigue las operaciones

Paso a paso de las operaciones:

1. push(4) → Pila: [4] (Tope: 4)
2. push(9) → Pila: [4, 9] (Tope: 9)
3. push(2) → Pila: [4, 9, 2] (Tope: 2)
4. pop() → Retira y devuelve **2**. Pila queda: [4, 9] (Tope: 9)
5. push(7) → Pila: [4, 9, 7] (Tope: 7)
6. peek() → Muestra el tope sin sacarlo. Devuelve **7**. Pila queda: [4, 9, 7] (Tope: 7)
7. pop() → Retira y devuelve **7**. Pila queda: [4, 9] (Tope: 9)
8. pop() → Retira y devuelve **9**. Pila queda: [4] (Tope: 4)

Diagrama estado por estado

Paso / Indicador	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
Operación	push(4)	push(9)	push(2)	pop()	push(7)	peek()	pop()	pop()
Contenido (Tope → Base)	[4]	[9,4]	[2,9,4]	[9,4]	[7,9,4]	[7,9,4]	[9,4]	[4]
Tope Actual	4	9	2	9	7	7	9	4
Resultado	–	–	–	devuelve 2	–	muestra 7	devuelve 7	devuelve 9

Elemento que queda al final: 4

Tope final: 4

2.2 Casos límite

1. **pop() sobre una pila vacía:** Ocurre un error conocido como desbordamiento negativo (*underflow*). Para manejarlo, revisamos antes de retirar si la pila está vacía mediante una condición. Si no tiene datos, mostramos un mensaje de aviso o retornamos **nulo** para evitar que el programa falle.
2. **peek() sobre una pila vacía:** Pasa lo mismo, no hay nada arriba que consultar. Lo manejamos verificando si la pila tiene elementos; si está vacía, devolvemos nulo o una alerta.
3. **Pseudocódigo para deshacer(pila):**

```
funcion deshacer(pila)
   si pila.estaVacia() entonces
      escribir "No hay acciones para deshacer."
      devolver nulo
   sino
      ultimaAccion = pila.pop()
      devolver ultimaAccion
   fin si
fin funcion
```

2.3 Aplicación: deshacer acciones

Secuencia realizada por el usuario: escribir → borrar → pegar.

Pila inicial (antes de deshacer):

```
|  pegar   | <- Tope (Última acción realizada)
|  borrar  |
| escribir | <- Base
```

Primer deshacer: Se retira pegar.

```
|  borrar  | <- Tope
| escribir | <- Base
```

Segundo deshacer: Se retira borrar.

```
| escribir | <- Tope y Base
```

¿Qué acción se deshace primero? Se deshace primero **pegar**.

Explicación: Una pila hace este comportamiento porque ella guarda las acciones una encima de la otra en el orden de llegada, cuando le damos deshacer, lo que

pasa es que se revierte la ultima accion que hicimos, osea la que esta en el tope de la fila.

Parte 3 — Colas (FIFO)

3.1 Simula la fila de atención

Paso a paso:

1. encolar("Ana") → Cola: [Ana]
2. encolar("Luis") → Cola: [Ana, Luis]
3. encolar("Sol") → Cola: [Ana, Luis, Sol]
4. desencolar() → Retira y devuelve a **Ana**. Queda: [Luis, Sol]
5. encolar("Mia") → Cola: [Luis, Sol, Mia]
6. desencolar() → Retira y devuelve a **Luis**. Queda: [Sol, Mia]
7. encolar("Teo") → Cola: [Sol, Mia, Teo]

Diagrama estado por estado:

- encolar("Ana"): [Frente: Ana] (Final: Ana)
- encolar("Luis"): [Frente: Ana] -> [Final: Luis]
- encolar("Sol"): [Frente: Ana] -> [Luis] -> [Final: Sol]
- desencolar(): Devuelve a "Ana". Queda: [Frente: Luis] -> [Final: Sol]
- encolar("Mia"): [Frente: Luis] -> [Sol] -> [Final: Mia]
- desencolar(): Devuelve a "Luis". Queda: [Frente: Sol] -> [Final: Mia]
- encolar("Teo"): [Frente: Sol] -> [Mia] -> [Final: Teo]

Resultados devueltos: El primer desencolar() devuelve a **Ana** y el segundo devuelve a **Luis**.

Personas que quedan esperando: Quedan **Sol**, **Mia** y **Teo** en ese orden (Sol en el frente, Teo en el final).

3.2 Operaciones y casos límite

1. **Pseudocódigo para atender(cola):**

```
funcion atender(cola)
   si cola.estaVacia() entonces
      escribir "No hay clientes en la fila."
      devolver nulo
   sino
      personaAtendida = cola.desencolar()
      devolver personaAtendida
   fin si
fin funcion
```

2. **Manejo de desencolar en cola vacía:** si necesitamos retirar algo cuando no hay nada en la cola, el programa sale como un error,entonces eso lo manejamos poniendo una condicion que pregunte si la cola esta vacia o no antes pues de intentar sacar o extraer lo que queremos.
3. **¿Por qué agregar al frente y retirar del final NO representa correctamente una fila FIFO si se mantienen esas operaciones?**
   si nosotros agregamos por el frente y retiramos por el final, invertimos los nombres pero vamos a seguir sacando las cosas que llegaron de primeras.

Parte 4 — Listas enlazadas simples

4.1 Construye y modifica una lista

1. **Insertar 18 al final:**
   Cabeza -> [18 | sig] -> nulo
2. **Insertar 7 al comienzo:**
   Cabeza -> [7 | sig] -> [18 | sig] -> nulo
3. **Insertar 24 al final:**
   Cabeza -> [7 | sig] -> [18 | sig] -> [24 | sig] -> nulo
4. **Insertar 13 después del nodo con 7:**
   Cabeza -> [7 | sig] -> [13 | sig] -> [18 | sig] -> [24 | sig] -> nulo
5. **Eliminar el nodo que contiene 18:**
   Cabeza -> [7 | sig] -> [13 | sig] -> [24 | sig] -> nulo

Diagrama final completo:

Cabeza -> [7 | sig] -> [13 | sig] -> [24 | sig] -> nulo

4.2 Diseña operaciones

1. **Recorrer la lista e imprimir todos sus datos:**

```
funcion recorrerEImprimir(cabeza)
   actual = cabeza
   mientras actual != nulo hacer
      escribir actual.dato
      actual = actual.siguiente
   fin mientras
fin funcion
```

2. **Buscar un valor y devolver si está presente:**

```
funcion buscar(cabeza, valorBuscado)
   actual = cabeza
   mientras actual != nulo hacer
      si actual.dato == valorBuscado entonces
         devolver Verdadero
      fin si
      actual = actual.siguiente
   fin mientras
```

```
    devolver Falso
fin funcion
```

3. **Insertar un nodo al comienzo:**

```
funcion insertarAlComienzo(cabeza, nuevoValor)
    nuevoNodo = crearNodo(nuevoValor)
    nuevoNodo.siguiente = cabeza
    cabeza = nuevoNodo
    devolver cabeza
fin funcion
```

4.3 Comprueba los bordes

Si la lista está vacía: cabeza apunta a nulo. La función le asigna a nuevoNodo.siguiente = nulo y luego actualiza cabeza para que apunte al nuevoNodo. si funciona.

Si la lista tiene nodos: nuevoNodo.siguiente guarda la dirección del antiguo primer nodo (que estaba en cabeza). Después, cabeza se actualiza para apuntar al nuevo nodo.

Referencias al eliminar el primer nodo: Se modifica únicamente la referencia **cabeza**, haciéndola apuntar directamente al segundo nodo (cabeza = cabeza.siguiente).

Diagrama antes y después de eliminar el primer nodo:

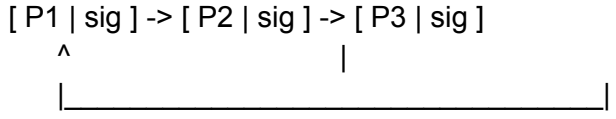
Antes de eliminar:
Cabeza -> [Nodo 1 | sig] -> [Nodo 2 | sig] -> [Nodo 3 | sig] -> nulo

Después de eliminar el Nodo 1:
Cabeza ----------> [Nodo 2 | sig] -> [Nodo 3 | sig] -> nulo

Parte 5 — Listas circulares

5.1 Reproduce un recorrido circular

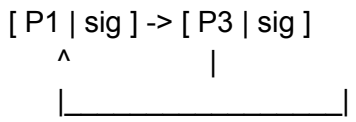
1. **Lista circular inicial:**



2. **Registro de los primeros 7 turnos:**
    - **Turno 1:** Atiende a **P1**. Siguiente en la lista: **P2**.
    - **Turno 2:** Atiende a **P2** (1.er turno de P2). Siguiente: **P3**.
    - **Turno 3:** Atiende a **P3**. Siguiente: **P1**.
    - **Turno 4:** Atiende a **P1**. Siguiente: **P2**.
    - **Turno 5:** Atiende a **P2** (2.º turno de P2 → **termina y se retira**). Siguiente: **P3**.

- **Turno 6:** Atiende a **P3**. Siguiente: **P1**.
- **Turno 7:** Atiende a **P1**. Siguiente: **P3**.

3. **Lista tras retirar P2:**

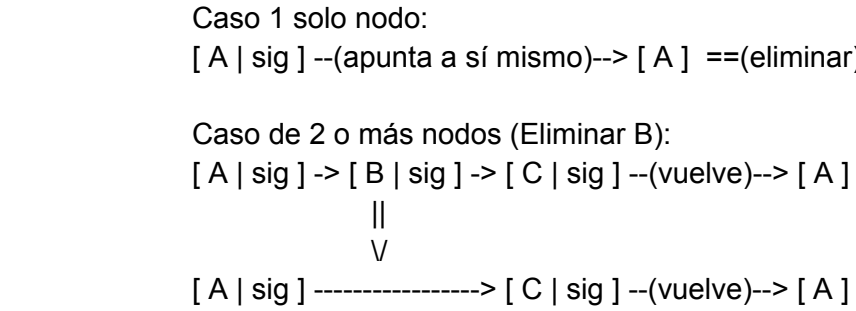


4. **Condición para detener el recorrido:** Guardamos la referencia del nodo en el que empezamos (nodoInicio = actual) y mientras vamos recorriendo la lista, la condicion que tenemos de parada es (actual != nodoInicio), ya cuando nos volvemos a encontar con el nodo que empezamos de primero nos damos cuenta que dimos la vuelta completa.

5.2 Casos especiales

1. **La lista está vacía:** No hay enlaces que modificar.
2. **La lista contiene un solo nodo:** El nodo apunta a sí mismo (nodo.siguiente = nodo).cuando lo eliminamos la primera referencia de la lista pasaria a ser un nulo.
3. **Se elimina el nodo considerado primero:** La referencia al principio de la lista pasa al segundo nodo y debemos recorrer la lista hasta el ultimo nodo para que podamos cambiar el enlace siguiente y hacer que apunte al nuevo nodo que esta de primero.
4. **Se elimina el último nodo:** recorremos hasta el penultimo nodo y actualizamos el siguiente enlace para que pueda apuntar hacia el primer nodo.

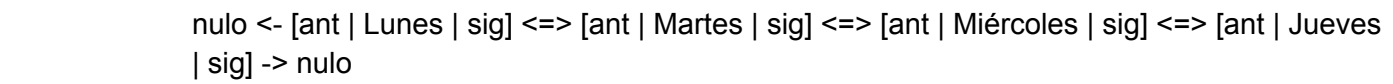
Diagramas de casos:



Parte 6 — Listas doblemente enlazadas

6.1 Recorrido en ambos sentidos

1. **Dibujo de la lista doblemente enlazada:**



2. **Recorrido de comienzo a final:** Lunes → Martes → Miércoles → Jueves.
3. **Recorrido de final a comienzo:** Jueves → Miércoles → Martes → Lunes.

6.2 Inserta y elimina

1. **Insertar Domingo antes de Lunes:**
   nulo <- [Domingo] <=> [Lunes] <=> [Martes] <=> [Miércoles] <=> [Jueves] -> nulo
2. **Insertar Viernes después de Jueves:**
   nulo <- [Domingo] <=> [Lunes] <=> [Martes] <=> [Miércoles] <=> [Jueves] <=> [Viernes] -> nulo
3. **Eliminar Miércoles:**
   nulo <- [Domingo] <=> [Lunes] <=> [Martes] <==========> [Jueves] <=> [Viernes] -> nulo

- **Referencias modificadas al eliminar Miércoles:**
  - El enlace siguiente del nodo **Martes** ahora apunta a **Jueves**.
  - El enlace anterior del nodo **Jueves** ahora apunta a **Martes**.

6.3 Compara los enlaces

Una ventaja de la lista doble es que podemos recorrerla hacia atrás sin tener errores,como cuando leímos los días desde jueves hasta lunes, algo que no podemos hacer directamente en una lista simple, Su costo es que requiere más memoria por guardar dos enlaces que serian el anterior y el siguiente en cada nodo, y si o si tenemos que actualizar el doble de punteros cada vez que insertamos o eliminamos elementos.

Parte 7 — Análisis y elección

Tabla de complejidad

Estructura y representación	Operación	Complejidad	Justificación breve
Pila enlazada con referencia al tope	Insertar en el tope	$O(1)$	solo se conecta el nodo como el nuevo tope en un solo paso.
Pila enlazada con referencia al tope	Retirar del tope	$O(1)$	quitamos el tope directo y movemos la referencia al siguiente..
Cola enlazada con referencias al frente y al final	Encolar al final	$O(1)$	con la referencia al final podemos pegar el nodo de una.

Cola enlazada con referencias al frente y al final	Desencolar del frente	$O(1)$	sacamos del frente y corremos la cabeza al nodo que sigue.
Lista simple sin referencia al último nodo	Insertar al comienzo	$O(1)$	enganchamos el nodo al inicio y no tenemos que tocar el resto de la lista.
Lista simple sin referencia al último nodo	Insertar al final	$O(n)$	si no tenemos referencia al final tenemos que recorrer todos los nodos que tengamos.
Lista doble con referencia a un nodo conocido	Desconectar ese nodo	$O(1)$	Con los enlaces anterior y siguiente lo unimos de una

Preguntas adicionales

1. **¿Por qué una cola enlazada que solo guarda el frente tarda más en encolar al final?**
   Porque al no tener guardada la ubicación exacta del último nodo siempre tiene que empezar a buscar desde el primer elemento y avanzar uno a uno hasta llegar al final para poder pegar el nuevo nodo, lo cual va tomar mas tiempo $O(n)$.
2. **¿Qué ventaja ofrece la lista circular para volver repetidamente al primer elemento?**
   nos ahorra tener que escribir condiciones que verifiquen manualmente para saber cuando llegamos al nulo, porque el ultimo nodo nos regresa de forma automatica al primero.
3. **¿Qué información adicional almacena una lista doble frente a una simple y qué permite hacer?**
   Almacena un enlace extra llamado anterior y esto nos ayuda o nos permite navegar hacia atras y tambien nos sirve para eliminar un nodo que conozcamos en tiempo $O(1)$ sin tener que recorrer toda la lista buscando quien estaba antes.

Parte 8 — Reflexión

Básicamente, entendí que según cómo conectemos los elementos, cambia mucho la forma y la rapidez con la que los metemos, buscamos o sacamos Por ejemplo en una pila manejamos todo por un solo lado, así que lo último que entra es lo primero que sale, y nos

sirve para eliminar acciones. tambien entendi que la cola funciona como una fila de una tienda, entramos por detras y salimos por la puerta de adelante para respetar los turnos, por ejemplo las listas no dan tanta cosa ,la simple solo avanza en un sentido,la circular se convierte como en un ciclo infinito porque siempre conecta la punta con el inicio, y en la lista doble nos deja movernos hacia adelante y hacia atras mas facil, aunque gastemos un poquitico mas de memoria.